PyLadies Rivers

Python Programming Curriculum (8 Weeks) — Beginner-Friendly Edition

Course Overview

This curriculum is built for people who have **never written a line of code before**. No assumptions are made about prior math, computer science, or technical background.

Every week follows the same rhythm so students always know what to expect:

1. **Concept explained in plain language** — before any code
2. **Code examples**, typed out and explained line by line
3. **One fully worked example** — solved completely, out loud, before students try anything themselves
4. **Practical exercises** — done live and then independently
5. **Common errors** — the specific mistakes beginners make at this stage, named in advance so they're not scary when they happen
6. **Assignment** — reinforces the week and, where relevant, builds on previous weeks

By the end of eight weeks, participants will be able to write basic Python programs confidently, and will be ready to continue into Data Analysis, AI, Web Development, Automation, or Software Engineering.

Week 1 – Getting Started with Python

1. What Is Programming? (5 min)

Say this on camera:

> "*A computer only does exactly what you tell it — nothing more, nothing less. Programming is the act of writing precise instructions, in a language the computer understands, so it does what you want. Python is one of those languages — and one of the friendliest to start with.*"

2. What Is Python? Brief History (5 min)

- Created by Guido van Rossum, first released in 1991
- Named after the comedy show *Monty Python's Flying Circus* — not the snake
- Known for readable, plain-English-like syntax, which is exactly why it's a great first language
- Used in web development, data science, AI, automation, and more

3. Brief History of PyLadies (3 min)

- PyLadies started in Los Angeles in 2011 to support women and gender minorities in the Python community
- Now a global network of chapters — PyLadies Rivers is one of them
- The mission: make tech feel welcoming, not intimidating

4. Installing Python & VS Code (10 min — hands-on, follow along live)

1. Go to **python.org** → Downloads → install the latest version
2. **Important on Windows:** tick "Add Python to PATH" during install
3. Go to **code.visualstudio.com** → install VS Code
4. Open VS Code → Extensions tab → search "Python" → install the official Microsoft extension

Common error: forgetting to check "Add to PATH" on Windows — this causes `'python' is not recognized` when typed in the terminal. Fix: reinstall and tick the box, or manually add Python to PATH.

5. The Python Interactive Shell (REPL) (5 min)

Open a terminal and type `python` (or `python3` on Mac/Linux). This drops you into a live shell where you can type code and see results immediately.

```
>>> 2 + 2
4
>>> print("Hello")
Hello
```

Explain: REPL stands for Read–Evaluate–Print–Loop. It's great for quick tests, but real programs are written in `.py` files — which is what we do next.

6. Writing and Running Your First Program (10 min)

In VS Code, create a new file named `hello.py`:

```
print("Hello, World!")
```

Save it, open the terminal in VS Code, and run:

```
python hello.py
```

Explain `print()` explicitly:

> "`print()` *is a built-in function that displays whatever is inside its parentheses on the screen. Anything in quotes is called a string — text data.*"

7. Python Syntax and Indentation (5 min)

> "*Python uses indentation (spaces at the start of a line) to know which lines of code belong together. Most other languages use curly braces* `{}` *for this — Python uses spacing instead. This matters from day one, so get comfortable with it now.*"

8. Writing Comments (3 min)

```
# This is a comment — Python ignores this line completely
print("This line actually runs")  # comments can also go at the end of a line
```

> "*Comments are notes to yourself or other humans reading the code. They don't affect how the program runs.*"

9. Reading Basic Error Messages (5 min)

Show a broken example live:

```
primt("Hello")
```

```
NameError: name 'primt' is not defined
```

Explain: Python reads errors bottom to top. The last line usually tells you *what* went wrong; the lines above show *where*. Reassure students: **every programmer, at every level, reads error messages constantly. This is normal, not a sign of failure.**

10. Worked Example (fully solved, before exercises)

Problem: Write a program that prints your name and your favourite hobby.

```
print("My name is Ada.")
print("My favourite hobby is reading.")
```

Walk through this line by line: two separate `print()` calls, each with its own string in quotes.

11. Practical Exercises

- Print your name
- Print your favourite hobby
- Print "Hello, World!"

12. Common Errors Recap

Error	Cause
`'python' is not recognized`	Python wasn't added to PATH during install
`NameError`	Misspelled a function name (e.g. `primt` instead of `print`)
`SyntaxError: EOL while scanning string literal`	Forgot a closing quote mark
Nothing prints	Forgot to call `print()` — just typing text does nothing on its own

Assignment

Write a program that displays your name, your age, your state, and your favourite food.

Week 2 – Variables, Data Types & Operators

1. What Is a Variable? (5 min)

> *"A variable is a named container that stores a value so you can use it later. Think of it like a labeled box — you put something inside, and you can look inside or swap what's there whenever you want."*

```
name = "Amaka"
age = 22
print(name)
print(age)
```

2. Naming Variables (3 min)

- Must start with a letter or underscore, not a number
- No spaces (use `first_name`, not `first name`)
- Case-sensitive: `Age` and `age` are different variables
- Use descriptive names — `age` is better than `x`

Common error:

```
2name = "Amaka"   # ✗ SyntaxError — can't start with a number
```

3. Data Types: Strings, Integers, Floats, Booleans (10 min)

```
name = "Amaka"        # string (str) — text, always in quotes
age = 22               # integer (int) — whole number
height = 1.65          # float — number with a decimal point
is_student = True      # boolean (bool) — True or False only
```

Show `type()` live:

```
print(type(name))   # <class 'str'>
print(type(age))    # <class 'int'>
```

4. Getting User Input (8 min)

```
name = input("What is your name? ")
print("Hello, " + name)
```

Important — flag this immediately: `input()` always returns a string, even if the user types a number.

```
age = input("Enter your age: ")
print(type(age))   # <class 'str'> — even though they typed "22"
```

5. Data Type Conversion (7 min)

```
age = input("Enter your age: ")
age = int(age)              # convert string to integer
print(age + 1)               # now this works
```

Common error:

```
age = input("Enter your age: ")
print(age + 1)   # ✗ TypeError: can only concatenate str (not "int") to str
```

Explain: you can't add a number to text — Python doesn't guess what you meant, so it errors instead.

6. Arithmetic, Assignment, and Comparison Operators (10 min)

Type	Examples
Arithmetic	`+ - * / //` (floor division) `%` (remainder) `**` (power)
Assignment	`= += -= *= /=`
Comparison	`== != > < >= <=`

```
score = 10
score += 5     # same as score = score + 5
print(score)   # 15
```

7. String Concatenation (5 min)

```
first = "Chioma"
last = "Eze"
full_name = first + " " + last
print(full_name)   # Chioma Eze
```

Also show f-strings as the cleaner modern approach:

```
print(f"{first} {last}")   # Chioma Eze
```

8. Worked Example

Problem: Ask the user for two numbers and print their sum.

```
num1 = input("Enter the first number: ")
num2 = input("Enter the second number: ")

num1 = int(num1)
num2 = int(num2)

total = num1 + num2
print(f"The sum is {total}")
```

Walk through why the conversion step is necessary before adding.

9. Practical Exercises

- Age Calculator
- Simple Calculator
- Temperature Converter

10. Common Errors Recap

Error	Cause
`TypeError: can only concatenate str`	Tried to add a string and a number without converting
`ValueError: invalid literal for int()`	Tried to convert non-numeric text (e.g. `int("abc")`)
Wrong variable name error	Typo, or case mismatch (`Name` vs `name`)

Assignment

Ask the user for their name and age, then tell them how old they will be in 10 years.

Week 3 – Decision Making with Python

(See full lecture note already prepared — reproduced here for a complete curriculum.)

1. Boolean Expressions (7 min)

```
print(5 > 3)     # True
is_raining = True
```

> *"A boolean expression is any statement that evaluates to `True` or `False`. Programs use booleans to make decisions."*

2. Comparison Operators (8 min)

Operator	Meaning
`==`	equal to
`!=`	not equal to
`> < >= <=`	greater/less than (or equal)

Common error:

```
if age = 18:      # ✗ single = is assignment, not comparison
```

3. `if`, `elif`, `else` Statements (15 min)

```
age = 16

if age >= 18:
    print("You are eligible to vote.")
elif age >= 16:
    print("You can register but must wait to vote.")
else:
    print("You are not yet eligible.")
```

> *"Python checks conditions top to bottom. As soon as one is `True`, it runs that block and skips the rest."*

4. Nested Conditions (8 min)

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("You may vote.")
    else:
        print("You need an ID to vote.")
else:
    print("You are not old enough to vote.")
```

5. Logical Operators — `and`, `or`, `not` (10 min)

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("You may vote.")
else:
    print("You cannot vote yet.")
```

> "`and` *needs both conditions true.* `or` *needs just one.* `not` *flips a boolean.*"

6. Worked Example

Problem: Check if a number is even or odd.

```
number = int(input("Enter a number: "))

if number % 2 == 0:
    print(f"{number} is even.")
else:
    print(f"{number} is odd.")
```

Explain `%` (modulo) as "the remainder after division."

7. Practical Exercises

- Even or Odd Checker
- Voting Eligibility Checker
- Grade Calculator

8. Common Errors Recap

Error	Cause
`SyntaxError`	Used `=` instead of `==`, or forgot the colon `:`
`IndentationError`	Inconsistent indentation
Wrong result, no crash	Used separate `if`s instead of `elif`, or mixed up `and`/`or`

Assignment

Accept a student's score and display the appropriate grade (A–F).

Week 4 – Loops

1. Why Loops Matter (5 min)

> *"Loops let you repeat an action without writing the same line of code over and over. If you wanted to print numbers 1 to 100, you would not want to write 100 `print()` lines — a loop does it in three."*

2. `while` Loops (8 min)

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Explain: the loop keeps running *while* the condition is `True`. Forgetting to update `count` causes an infinite loop.

Common error — the infinite loop:

```
count = 1
while count <= 5:
    print(count)   # ✗ forgot count += 1 — this runs forever
```

Show students how to stop a runaway program: `Ctrl + C` in the terminal.

3. `for` Loops and `range()` (10 min)

```
for i in range(1, 6):
    print(i)
```

> *"`range(1, 6)` produces the numbers 1 through 5 — it stops before the second number. `for` loops are ideal when you know how many times you want to repeat something."*

```
for i in range(5):
    print(i)   # 0, 1, 2, 3, 4 — range starts at 0 by default
```

4. Nested Loops (7 min)

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i*j}")
```

Explain: the inner loop completes fully for every single pass of the outer loop.

5. `break` and `continue` (7 min)

```
for i in range(1, 10):
    if i == 5:
        break          # stop the loop entirely
    print(i)
```

```
for i in range(1, 6):
    if i == 3:
        continue       # skip this iteration, keep looping
    print(i)
```

6. Worked Example

Problem: Print the multiplication table for the number 5.

```
number = 5

for i in range(1, 13):
    print(f"{number} x {i} = {number * i}")
```

Walk through this fully before assigning tables 1–12.

7. Practical Exercises

- Multiplication Table Generator
- Number Guessing Game
- Countdown Timer

8. Common Errors Recap

Error	Cause
Program never stops	Forgot to update the loop variable in a `while` loop
`range()` gives one fewer number than expected	`range(1, 6)` stops *before* 6, i.e. gives 1–5
Loop skips or repeats unexpectedly	`break`/`continue` placed in the wrong spot, or wrong indentation level

Assignment

Print the multiplication tables from 1 to 12.

Week 5 – Functions

(Full lecture note already prepared — reproduced here for a complete curriculum.)

1. Why Functions Are Useful (5 min)

> *"A function is a reusable block of code that performs one specific task. Instead of repeating code, write it once, name it, and call it whenever needed."*

2. Creating Functions with `def` (8 min)

```
def greet():
    print("Hello, welcome to PyLadies Rivers!")

greet()
```

3. Parameters and Arguments (10 min)

```
def greet(name):
    print(f"Hello, {name}! Welcome to PyLadies Rivers.")

greet("Amaka")
```

> *"Parameter = the placeholder in the definition. Argument = the actual value you pass in."*

4. Default Parameters (7 min)

```
def greet(name="friend"):
    print(f"Hello, {name}!")

greet()          # Hello, friend!
greet("Ngozi")   # Hello, Ngozi!
```

5. Returning Values with `return` (10 min)

```
def add(a, b):
    return a + b

result = add(3, 5)
print(result)     # 8
```

> *"`print()` only displays something. `return` sends a value back out so you can store and reuse it."*

Common error:

```
def add(a, b):
    print(a + b)

result = add(3, 5)    # result is None!
```

6. Local and Global Variables (8 min)

```
x = 10

def show_number():
    x = 5
    print(x)

show_number()   # 5
print(x)        # 10 — unaffected
```

7. Worked Example

Problem: Find the largest of two numbers.

```
def largest(a, b):
    if a > b:
        return a
    else:
        return b

print(largest(15, 9))   # 15
```

8. Practical Exercises

- Add two numbers
- Find the largest of three numbers
- Calculate the area of a rectangle

9. Common Errors Recap

Error	Cause
`TypeError: missing required positional argument`	Called function with too few arguments
Function "does nothing" with its result	Used `print()` instead of `return`
`NameError`	Used a local variable outside its function

Assignment

Build a menu-driven calculator using functions.

Week 6 – Data Structures

1. Lists (10 min)

```
fruits = ["apple", "banana", "mango"]
print(fruits[0])       # apple
fruits.append("kiwi")
print(fruits)
```

> *"A list stores multiple values in one variable, in order. Indexing starts at 0, not 1 — this trips up everyone at first, so don't worry if it feels odd."*

2. Tuples (5 min)

```
coordinates = (6.5, 3.4)
print(coordinates[0])
```

> *"A tuple is like a list, but it cannot be changed after creation — useful for values that should stay fixed."*

Common error:

```
coordinates[0] = 10   # ✗ TypeError: 'tuple' object does not support item assignment
```

3. Dictionaries (10 min)

```
student = {"name": "Amaka", "age": 22, "course": "Python"}
print(student["name"])    # Amaka
student["age"] = 23        # update a value
```

> *"A dictionary stores data in key-value pairs — instead of a numbered position, you look things up by a name (the key)."*

4. Sets (5 min)

```
numbers = {1, 2, 3, 3, 2}
print(numbers)   # {1, 2, 3} — duplicates automatically removed
```

5. Indexing and Slicing (8 min)

```
fruits = ["apple", "banana", "mango", "kiwi"]
print(fruits[1])        # banana
print(fruits[-1])        # kiwi — negative index counts from the end
print(fruits[1:3])       # ['banana', 'mango'] — slice, stops before index 3
```

6. Nested Lists and Dictionaries (7 min)

```
students = [
    {"name": "Amaka", "age": 22},
    {"name": "Chioma", "age": 24}
]
print(students[0]["name"])   # Amaka
```

7. Choosing the Right Data Structure (5 min)

Use this...	...when you need
List	An ordered collection you'll change often

Use this...	...when you need
Tuple	An ordered collection that shouldn't change
Dictionary	Data linked by a name/label (key)
Set	A collection with no duplicates, order doesn't matter

8. Worked Example

Problem: Store three fruits in a list and print each one with a loop.

```
fruits = ["apple", "banana", "mango"]

for fruit in fruits:
    print(f"I like {fruit}")
```

9. Practical Exercises

- Student Record System
- Contact Book
- Shopping List Manager

10. Common Errors Recap

Error	Cause
`IndexError: list index out of range`	Tried to access an index that doesn't exist
`KeyError`	Tried to access a dictionary key that doesn't exist
`TypeError` on tuple	Tried to modify a tuple after creation
Off-by-one confusion	Forgot indexing starts at 0, and slicing stops *before* the end index

Assignment

Store the details of five students using dictionaries and display their information.

Week 7 – Files, Modules & Libraries

1. Reading and Writing Text Files (10 min)

```
# Writing to a file
with open("notes.txt", "w") as file:
    file.write("Hello, this is my first file!")

# Reading from a file
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)
```

> "`with open(...) as file:` *automatically closes the file for you when done — always prefer this pattern over opening a file manually.*"

Common error:

```
with open("notes.txt", "r") as file:   # ✗ FileNotFoundError if the file doesn't exist y
    et
```

Explain: `"w"` mode creates a file if missing (and overwrites it if it exists); `"r"` mode requires the file to already exist.

2. Working with CSV Files (8 min)

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Score"])
    writer.writerow(["Amaka", 85])
```

3. Importing Modules (5 min)

```
import math
print(math.sqrt(16))   # 4.0
```

> "*A module is a file full of pre-written code that you can reuse instead of writing everything from scratch.*"

4. Built-in Modules: `math`, `random`, `datetime` (8 min)

```
import random
print(random.randint(1, 10))   # random number between 1 and 10

import datetime
print(datetime.date.today())
```

5. Installing Packages with `pip` (5 min)

```
pip install pandas
```

> "`pip` *is Python's package installer — it downloads code other people have written so you don't reinvent the wheel.*"

6. Introduction to `pandas` (7 min)

```
import pandas as pd

data = {"Name": ["Amaka", "Chioma"], "Score": [85, 90]}
df = pd.DataFrame(data)
print(df)
```

7. Worked Example

Problem: Write a list of scores to a CSV file, then read it back and display it.

```
import csv

scores = [["Amaka", 85], ["Chioma", 90]]

with open("scores.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Score"])
    writer.writerows(scores)

with open("scores.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

8. Practical Exercises

- Random Password Generator
- Student Score Recorder
- CSV Reader

9. Common Errors Recap

Error	Cause
`FileNotFoundError`	Tried to read a file that doesn't exist, or wrong file path
`ModuleNotFoundError`	Package not installed — run `pip install <package>`
Data looks like one long string	Forgot `newline=""` when writing CSVs on Windows, causing extra blank rows

Assignment

Create a program that stores students' scores in a CSV file and displays the contents.

Week 8 – Capstone Project

Final Project

Each participant builds a complete command-line application that demonstrates everything learned so far:

- Variables
- User input
- Operators
- Conditional statements
- Loops
- Functions
- Data structures
- File handling
- Modules

How to Approach It (10 min guidance, not code-heavy)

1. **Pick one idea** from the list below — don't overthink this
2. **Sketch the flow on paper first**: what does the menu look like? What does each option do?
3. **Build it piece by piece**: get one function working before adding the next
4. **Test constantly** — run the program after every small change, not just at the end
5. **Save data to a file** so information isn't lost when the program closes

Suggested Project Ideas

- Student Grade Tracker
- Expense Tracker
- Market Price Logger
- Personal Contact Book
- Patient Record Manager
- Library Management System
- PyLadies Rivers Member Directory

Minimal Structure to Reassure Beginners

```
def main_menu():
    while True:
        print("1. Add Record  2. View Records  3. Exit")
        choice = input("Choose an option: ")

        if choice == "1":
            pass   # call an "add" function here
        elif choice == "2":
            pass   # call a "view" function here
        elif choice == "3":
            break
        else:
            print("Invalid choice, try again.")

main_menu()
```

> "*This skeleton alone already uses loops, conditionals, functions, and input — you're not starting from zero. You're assembling pieces you already know.*"

Participants present their projects to the class during the final session.

Weekly Learning Activities

Every training session includes:

- Interactive Lecture
- Live Coding Demonstration
- Hands-on Practical Exercises
- Weekly Assignment
- Question & Answer Session
- Mentor Support

Learning Outcomes

By the end of this course, participants will be able to:

- Understand the fundamentals of programming
- Write and run Python programs
- Work with variables and different data types
- Accept user input and process data
- Make decisions using conditional statements
- Automate repetitive tasks with loops
- Write reusable functions
- Work with Python data structures
- Read from and write to files
- Import and use Python modules and libraries
- Build beginner-friendly command-line applications

Beyond the 8 Weeks

After completing this program, participants will be introduced to:

- Git & GitHub
- Open Source Contributions
- Object-Oriented Programming (OOP)
- SQL Fundamentals
- Data Analysis with Pandas
- Backend Development with Python
- APIs
- DevOps Fundamentals
- Artificial Intelligence & Machine Learning
- AI Solutions Engineering
- Career Development in Technology

Ada's Reminder

> "*You don't need a computer science background to succeed in tech. Every expert was once a beginner. Learn one step at a time, ask questions, practice consistently, and celebrate every small win. At PyLadies Rivers, we learn together, support one another, and believe that everyone belongs in technology.*"